

# EN3160 Assignment 1

## Intensity Transformations and Neighborhood Filtering

**Name:** Kumarasinghe M. N.

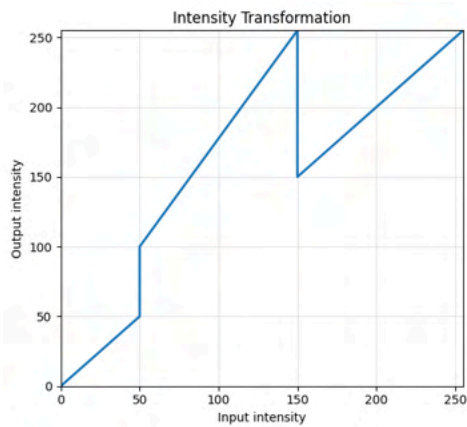
**Index Number:** 230355X

**Repository:** [https://github.com/NethsanduKumarasinghe/EN3160\\_Assignment\\_01.git](https://github.com/NethsanduKumarasinghe/EN3160_Assignment_01.git)

```
In [15]: import cv2 as cv #Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
```

### Question 1

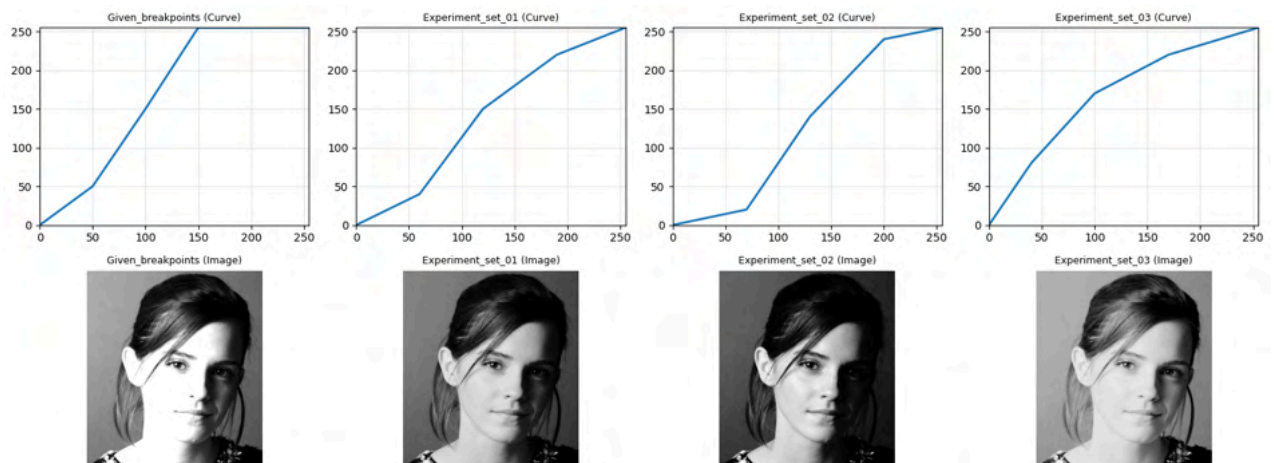
```
In [16]: image_path = "/content/images/q1.jpeg"
im = cv.imread(image_path, cv.IMREAD_GRAYSCALE)
assert im is not None
def intensity_transform(im, breakpoints): # Defining the function for intensity transformation
    output = np.zeros_like(im, dtype=np.float32)
    for i in range(len(breakpoints) - 1):
        x1, y1 = breakpoints[i]
        x2, y2 = breakpoints[i + 1]
        if x1 == x2:
            continue
        mask = (im >= x1) & (im < x2)
        output[mask] = y1 + (im[mask] - x1) * (y2 - y1) / (x2 - x1)
    output[im >= breakpoints[-1, 0]] = breakpoints[-1, 1]
    return np.clip(output, 0, 255).astype(np.uint8)
# breakpoints related to figure 1a
breakpoints = np.array([[0, 0], [50, 50], [50, 100], [150, 255], [150, 150], [255, 255]])
transformed = intensity_transform(im, breakpoints)
plt.figure(figsize=(15, 5)) # Plotting
plt.subplot(1, 3, 1)
x_plot = breakpoints[:, 0]
y_plot = breakpoints[:, 1]
plt.plot(x_plot, y_plot, linewidth=2)
plt.xlim(0, 255)
plt.ylim(0, 255)
plt.xlabel("Input intensity")
plt.ylabel("Output intensity")
plt.title("Intensity Transformation")
plt.grid(alpha=0.3)
plt.subplot(1, 3, 2)
plt.imshow(im, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(transformed, cmap="gray", vmin=0, vmax=255)
plt.title("Transformed Image")
plt.axis("off")
plt.tight_layout()
plt.show()
```



## Visually pleasing image with different breakpoints

```
In [17]: # breakpoints for experiment and the give breakpoint set in the assignment
breakpoint_experiments = [
    ("Given_breakpoints", np.array([[0, 0], [50, 50], [100, 150], [150, 255], [255, 255]])),
    ("Experiment_set_01", np.array([[0, 0], [60, 40], [120, 150], [190, 220], [255, 255]])),
    ("Experiment_set_02", np.array([[0, 0], [70, 20], [130, 140], [200, 240], [255, 255]])),
    ("Experiment_set_03", np.array([[0, 0], [40, 80], [100, 170], [170, 220], [255, 255]]))
]

plt.figure(figsize=(16, 6)) #plotting
for i, (name, bp) in enumerate(breakpoint_experiments):
    transformed = intensity_transform(im, bp)
    # Plot intensity transfer curve
    plt.subplot(2, 4, i + 1)
    plt.plot(bp[:, 0], bp[:, 1], linewidth=2)
    plt.xlim(0, 255)
    plt.ylim(0, 255)
    plt.title(f"{name} (Curve)", fontsize=9)
    plt.grid(alpha=0.3)
    # Plot transformed image
    plt.subplot(2, 4, i + 5)
    plt.imshow(transformed, cmap="gray", vmin=0, vmax=255)
    plt.title(f"{name} (Image)", fontsize=9)
    plt.axis("off")
plt.tight_layout()
plt.show()
```



## Discussion: Question 1 – Intensity Transformations

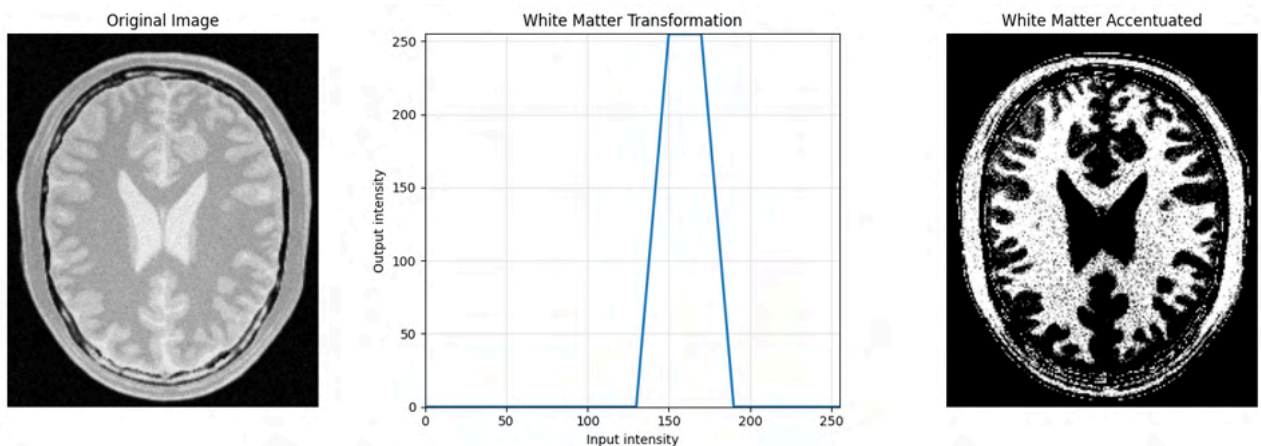
- **Piecewise Function:** The custom implementation successfully maps pixel intensities using specified control points, allowing for targeted, non-linear contrast adjustments.
- **Analysis of Given Breakpoints in Fig. 1a:** The default breakpoint set contains vertical discontinuities (e.g., jumping from output 50 to 100 at input 50). This abrupt mapping causes severe posterization and unnatural contour artifacts, destroying smooth gradients on the subject's face.
- **Experimental Breakpoint Sets:** The three custom sets utilize continuous, monotonically increasing control points. These configurations successfully enhance image contrast and brightness while maintaining natural tonal

transitions and completely avoiding banding artifacts.

## Question 2

### a) Accentuate white matter

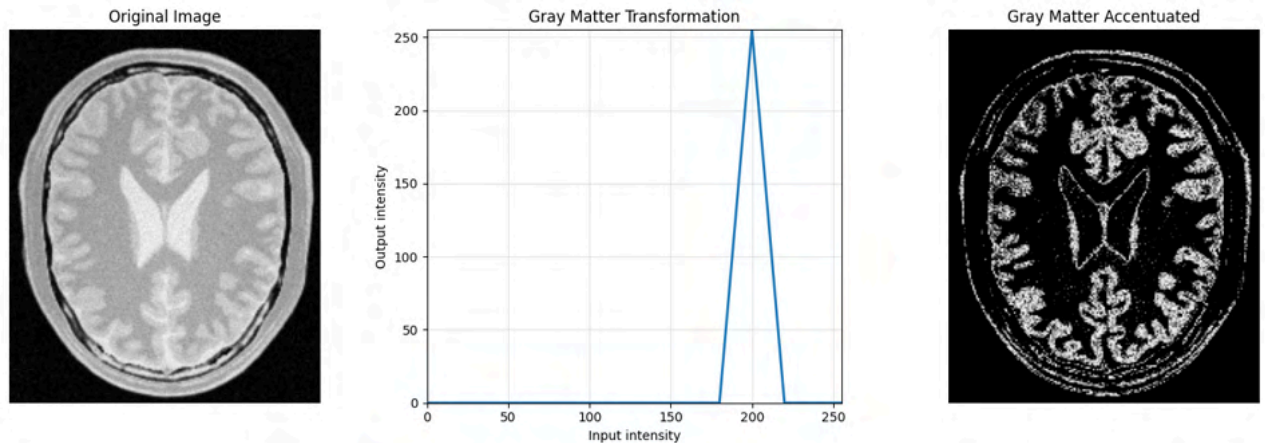
```
In [18]: image_path = "/content/images/q2.jpeg"
im = cv.imread(image_path, cv.IMREAD_GRAYSCALE)
assert im is not None
# required breakpoints
white_breakpoints = np.array([[0, 0], [130, 0], [150, 255], [170, 255], [190, 0], [255, 0]])
white_matter = intensity_transform(im, white_breakpoints) #using the same function defined in quest
plt.figure(figsize=(15, 5)) #plotting
plt.subplot(1, 3, 1)
plt.imshow(im, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.plot(white_breakpoints[:, 0], white_breakpoints[:, 1], linewidth=2)
plt.xlim(0, 255)
plt.ylim(0, 255)
plt.xlabel("Input intensity")
plt.ylabel("Output intensity")
plt.title("White Matter Transformation")
plt.grid(alpha=0.3)
plt.subplot(1, 3, 3)
plt.imshow(white_matter, cmap="gray", vmin=0, vmax=255)
plt.title("White Matter Accentuated")
plt.axis("off")
plt.tight_layout()
plt.show()
```



### b) Accentuate gray matter

```
In [19]: #required breakpoints
gray_breakpoints = np.array([[0, 0], [180, 0], [200, 255], [220, 0], [255, 0]])
gray_matter = intensity_transform(im, gray_breakpoints) #transformation
plt.figure(figsize=(15, 5)) #plotting
plt.subplot(1, 3, 1)
plt.imshow(im, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.plot(gray_breakpoints[:, 0], gray_breakpoints[:, 1], linewidth=2)
plt.xlim(0, 255)
plt.ylim(0, 255)
plt.xlabel("Input intensity")
plt.ylabel("Output intensity")
plt.title("Gray Matter Transformation")
plt.grid(alpha=0.3)
plt.subplot(1, 3, 3)
plt.imshow(gray_matter, cmap="gray", vmin=0, vmax=255)
plt.title("Gray Matter Accentuated")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



## Discussion: Question 2 – Brain Tissue Accentuation

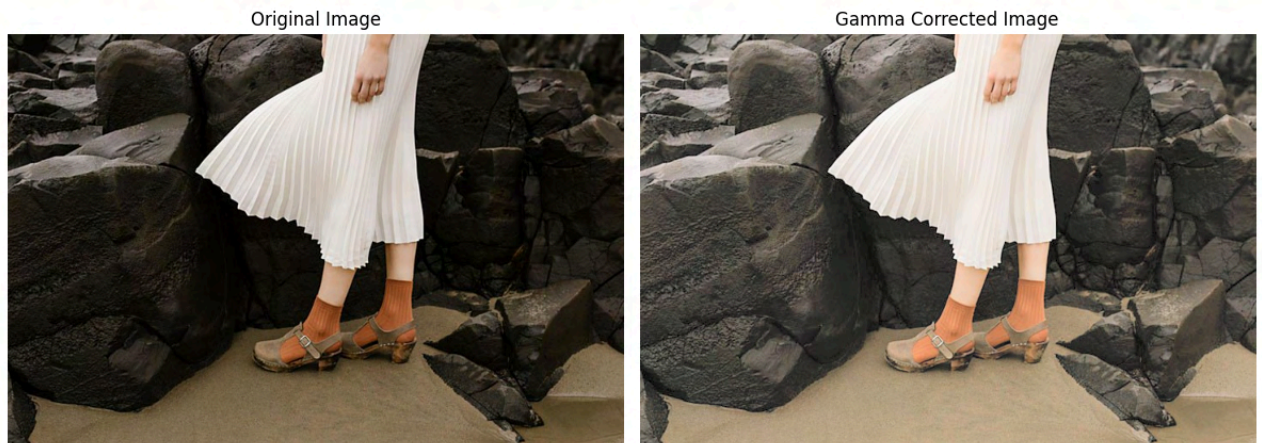
- **Intensity Slicing Strategy:** The operation applied piecewise linear transformations to accentuate specific tissues, specifically white matter and gray matter, within the brain proton density image.
- **White Matter Isolation:** The transformation isolated a specific mid-level intensity band by mapping input intensities between 130 and 190 to high output values (peaking at 255 for inputs between 150 and 170) while setting all other values to 0. This effectively masked out non-target tissues, leaving the distinct central white matter structures prominently illuminated.
- **Gray Matter Isolation:** A narrower transformation was applied to target a higher intensity range, mapping input values between 180 and 220 to high output values (peaking at 255 for an input of 200) and suppressing everything else to 0. This successfully segmented the outer cortical folds (gray matter) while completely blacking out the white matter and background.
- **Conclusion:** By defining distinct, localized breakpoint windows, we can perform effective, threshold-based medical image segmentation that isolates the unique pixel intensity signatures of different anatomical structures.

## Question 3

a)

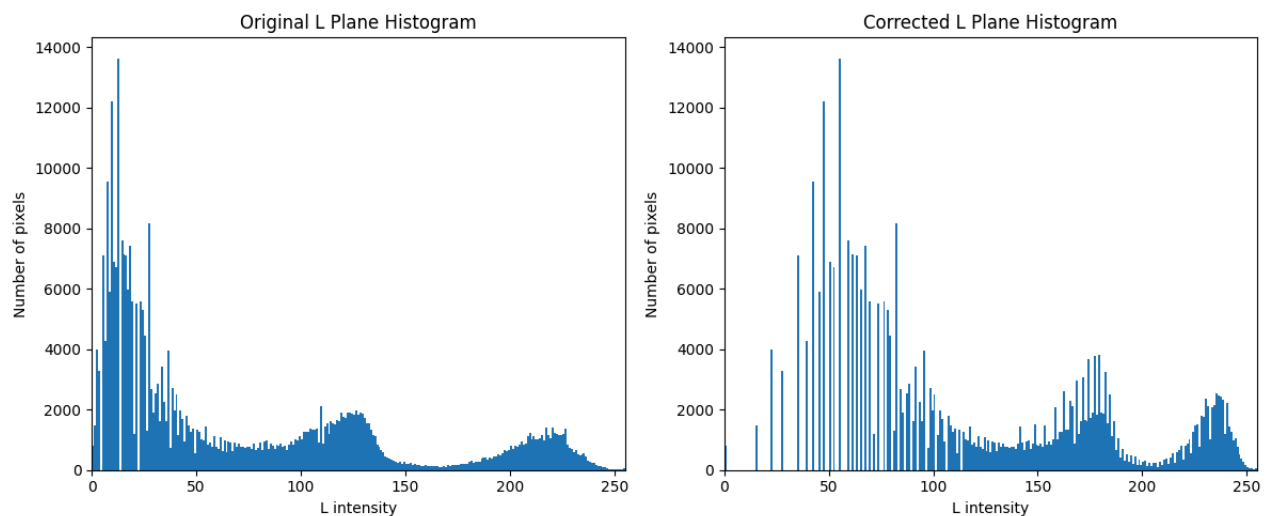
Used gamma value = 0.5

```
In [20]: image_path = "/content/images/q3.jpeg"
im = cv.imread(image_path)
assert im is not None
lab = cv.cvtColor(im, cv.COLOR_BGR2LAB)
L, a, b = cv.split(lab) #splitting
gamma = 0.5 #stating gamma value
table = np.array([((i / 255.0) ** gamma) * 255 for i in range(256)]).astype("uint8")
L_corrected = cv.LUT(L, table) #applying gamma correction to L plane
lab_corrected = cv.merge([L_corrected, a, b])
corrected = cv.cvtColor(lab_corrected, cv.COLOR_LAB2BGR)
plt.figure(figsize=(12, 5)) #plotting
plt.subplot(1, 2, 1)
plt.imshow(cv.cvtColor(im, cv.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(cv.cvtColor(corrected, cv.COLOR_BGR2RGB))
plt.title("Gamma Corrected Image")
plt.axis("off")
plt.tight_layout()
plt.show()
```



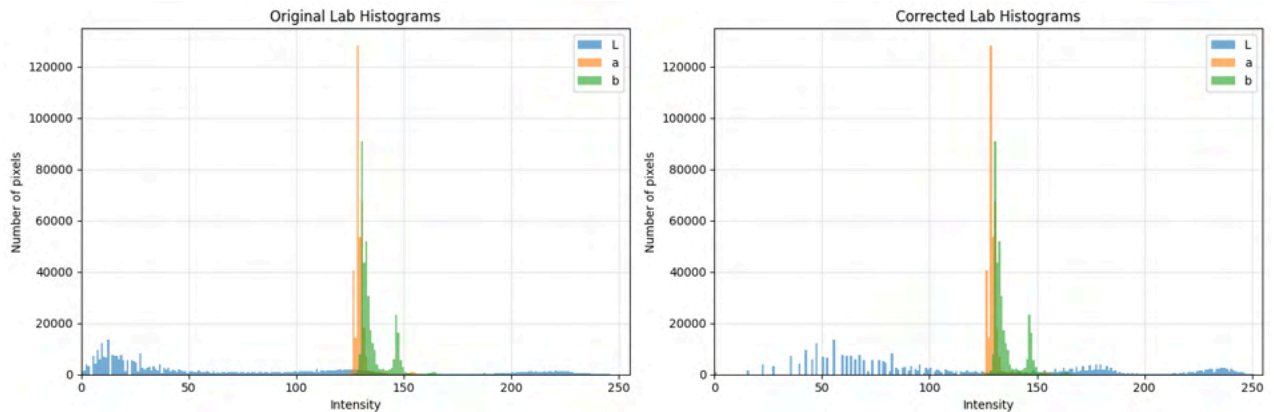
b)

```
In [21]: plt.figure(figsize=(12, 5)) #plotting histograms asked in part b
plt.subplot(1, 2, 1)
plt.hist(L.ravel(), bins=256, range=(0, 256))
plt.xlabel("L intensity")
plt.ylabel("Number of pixels")
plt.title("Original L Plane Histogram")
plt.xlim(0, 255)
plt.subplot(1, 2, 2)
plt.hist(L_corrected.ravel(), bins=256, range=(0, 256))
plt.xlabel("L intensity")
plt.ylabel("Number of pixels")
plt.title("Corrected L Plane Histogram")
plt.xlim(0, 255)
plt.tight_layout()
plt.show()
```



```
In [22]: plt.figure(figsize=(15, 5))
plt.subplot(1, 2, 1)
plt.hist(L.ravel(), bins=256, range=(0, 256), alpha=0.6, label="L")
plt.hist(a.ravel(), bins=256, range=(0, 256), alpha=0.6, label="a")
plt.hist(b.ravel(), bins=256, range=(0, 256), alpha=0.6, label="b")
plt.xlabel("Intensity")
plt.ylabel("Number of pixels")
plt.title("Original Lab Histograms")
plt.xlim(0, 255)
plt.legend()
plt.grid(alpha=0.3)
plt.subplot(1, 2, 2)
plt.hist(L_corrected.ravel(), bins=256, range=(0, 256), alpha=0.6, label="L")
plt.hist(a.ravel(), bins=256, range=(0, 256), alpha=0.6, label="a")
plt.hist(b.ravel(), bins=256, range=(0, 256), alpha=0.6, label="b")
plt.xlabel("Intensity")
plt.ylabel("Number of pixels")
plt.title("Corrected Lab Histograms")
plt.xlim(0, 255)
```

```
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

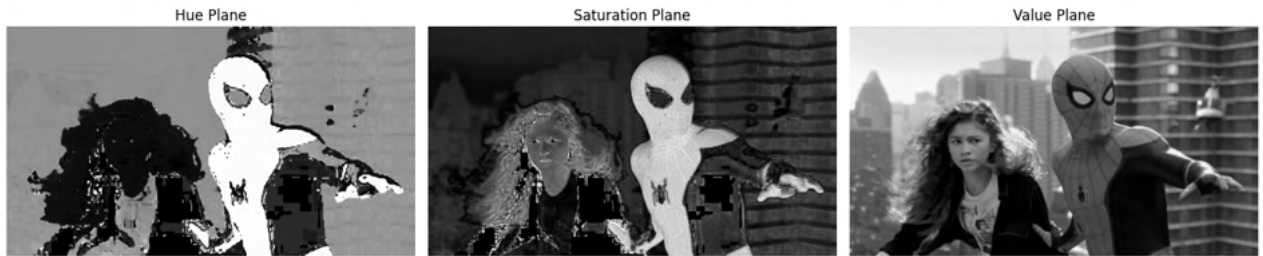


## Discussion: Question 3 – Gamma Correction in $L^*a^*b^*$ Color Space

- **Transformation & Gamma Value:** A gamma correction of  $\gamma = 0.5$  was selected and applied exclusively to the Lightness ( $L$ ) channel.
- **Visual Enhancement:** Because a fractional gamma value ( $\gamma < 1$ ) applies a non-linear brightening effect, the transformation expands the contrast in the darker regions. This successfully reveals hidden textures and details in the dark rocks without blowing out the highlights in the white dress.
- **Histogram Analysis ( $L$  Plane):** The original  $L$  histogram is heavily skewed toward the left, indicating a dominance of dark pixels. Following the transformation, the  $L$  histogram spreads outward and shifts toward the mid-to-high intensity ranges, quantitatively confirming the overall brightening effect.
- **Color Preservation:** Performing this correction in the  $L^*a^*b^*$  color space rather than standard RGB ensures that chromaticity is isolated. The combined Lab histograms prove that the  $a$  and  $b$  color channels remain completely untouched, increasing image brightness without introducing unnatural color shifts or saturation artifacts.

## Question 4

```
In [23]: image_path = "/content/images/q4.jpeg"
im = cv.imread(image_path)
assert im is not None
im_rgb = cv.cvtColor(im, cv.COLOR_BGR2RGB)
hsv = cv.cvtColor(im, cv.COLOR_BGR2HSV)
H, S, V = cv.split(hsv) #splitting to H,S,V planes
plt.figure(figsize=(15, 5)) #displaying the planes
plt.subplot(1, 3, 1)
plt.imshow(H, cmap="gray")
plt.title("Hue Plane")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(S, cmap="gray")
plt.title("Saturation Plane")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(V, cmap="gray")
plt.title("Value Plane")
plt.axis("off")
plt.tight_layout()
plt.show()
```



```
In [24]: x = S.astype(np.float64)
sigma = 70.0
a = 0.75 #after trying different values selected a=0.75
exponent = -((x - 128.0)**2) / (2.0 * sigma**2)
modifier = a * 128.0 * np.exp(exponent)
#applying the transformation
S_enhanced = np.clip(x + modifier, 0, 255).astype(np.uint8)
hsv_enhanced = cv.merge([H, S_enhanced, V])
im_vibrant = cv.cvtColor(hsv_enhanced, cv.COLOR_HSV2RGB) #recombining three planes
x_vals = np.arange(0, 256)
y_vals = np.clip(x_vals + a * 128.0 * np.exp(-((x_vals - 128.0)**2) / (2.0 * sigma**2)), 0, 255)
plt.figure(figsize=(18, 5)) #display the required figures
plt.subplot(1, 3, 1)
plt.imshow(im_rgb)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(im_vibrant)
plt.title(f"Vibrance-Enhanced (a = {a})")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.plot(x_vals, y_vals, color='blue', linewidth=2)
plt.plot(x_vals, x_vals, color='gray', linestyle='---')
plt.title("Intensity Transformation Curve")
plt.xlabel("Input Intensity (x)")
plt.ylabel("Output Intensity (f(x))")
plt.xlim([0, 255])
plt.ylim([0, 255])
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



### (c) Parameter Selection

**Reported Value:** `a = 0.75`

**Justification:** Setting `a = 0.75` provides a visually pleasing enhancement to the image's vibrance. This value is high enough to significantly boost the mid-tone saturations (making the reds and blues of the suit pop) while remaining low enough to prevent unnatural oversaturation or severe color clipping at the high end of the spectrum.

### Discussion: Question 4 – Vibrance Enhancement

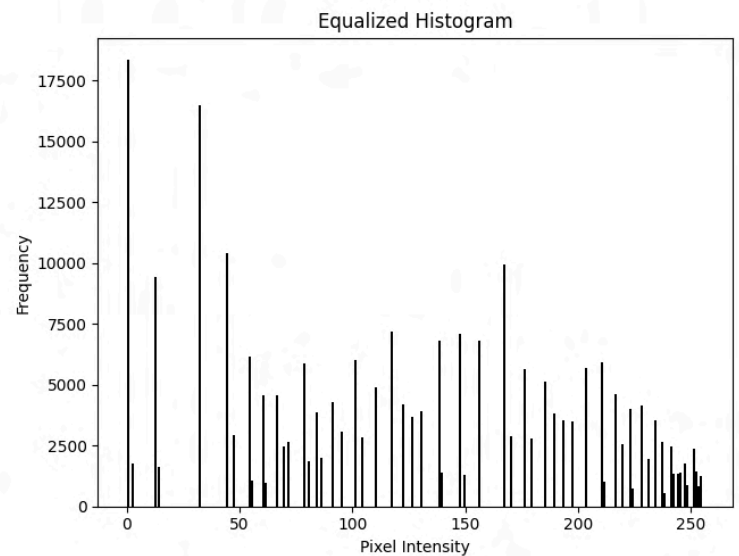
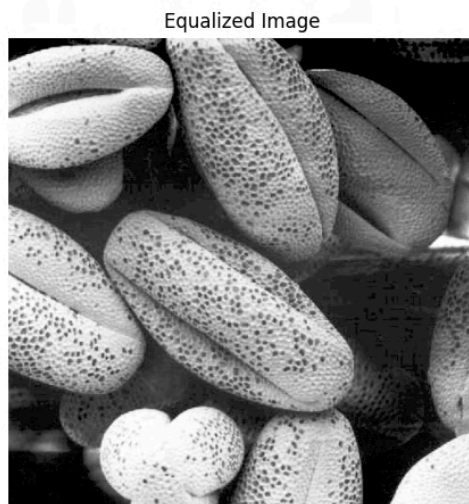
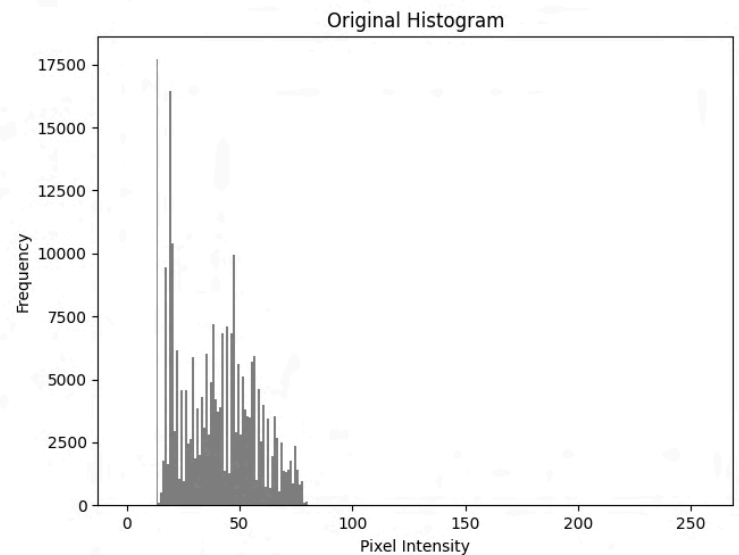
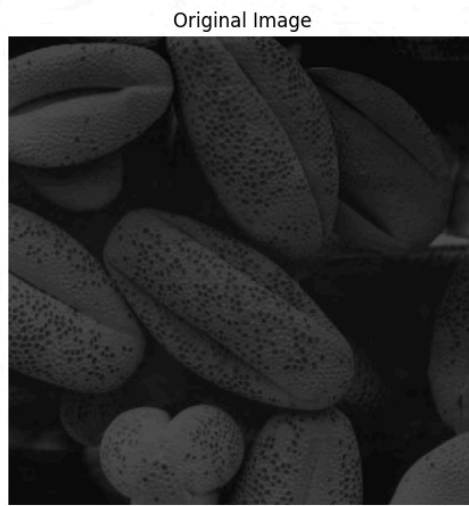
- **Targeted Channel Processing:** By converting the image to the HSV color space and applying the transformation exclusively to the Saturation plane, the operation successfully increases color intensity without altering the underlying brightness or base colors.

- **Non-Linear Mid-Tone Boost:** The applied mathematical function  $f(x) = \min(x + a \times 128e^{-\frac{(x-128)^2}{2\sigma^2}}, 255)$  uses a Gaussian-weighted boost centered at an input intensity of 128. This intelligently targets mid-level saturated pixels for the maximum boost, while tapering off near 0 and 255 to prevent already highly saturated colors from clipping.
- **Visual Result:** Setting  $a = 0.75$  provides a visually pleasing enhancement that significantly boosts the mid-tone saturations, making the reds and blues of the suit pop. This value remains low enough to prevent unnatural oversaturation or severe color clipping at the high end of the spectrum.

## Question 5

```
In [25]: def custom_histogram_equalization(img): #custom function for histogram equalization
hist, bins = np.histogram(img.flatten(), 256, [0, 256])
cdf = hist.cumsum()
cdf_masked = np.ma.masked_equal(cdf, 0)
cdf_normalized = (cdf_masked - cdf_masked.min()) * 255 / (cdf_masked.max() - cdf_masked.min())
cdf_final = np.ma.filled(cdf_normalized, 0).astype(np.uint8)
img_equalized = cdf_final[img]
return img_equalized

image_path = "/content/images/q5.tif"
img = cv.imread(image_path, cv.IMREAD_GRAYSCALE)
assert img is not None
img_eq = custom_histogram_equalization(img)
plt.figure(figsize=(12, 10)) #plotting
plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray', vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.hist(img.flatten(), bins=256, range=[0, 256], color='gray')
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.subplot(2, 2, 3)
plt.imshow(img_eq, cmap='gray', vmin=0, vmax=255)
plt.title("Equalized Image")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.hist(img_eq.flatten(), bins=256, range=[0, 256], color='black')
plt.title("Equalized Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



## Discussion: Question 5 – Custom Histogram Equalization

- **Custom Implementation:** The defined `custom_histogram_equalization` function successfully maps pixel values by calculating the cumulative distribution function (CDF) and normalizing it to span the full 8-bit intensity range (0–255).
- **Original State:** The original image is severely underexposed and lacks dynamic range. This is quantitatively confirmed by the original histogram, where pixel intensities are densely compressed into the darker, lower-value bins (primarily between 0 and 80).
- **Visual Enhancement:** Applying the equalization process drastically improves global contrast, pulling out fine microscopic textures and structural details on the surfaces of the pollen grains that were previously hidden in shadows.
- **Histogram Comparison:** The equalized histogram proves the effectiveness of the transformation. The pixel intensities are now broadly stretched and distributed across the entire 0–255 spectrum, validating that the custom CDF mapping achieved a much flatter and mathematically balanced intensity distribution.

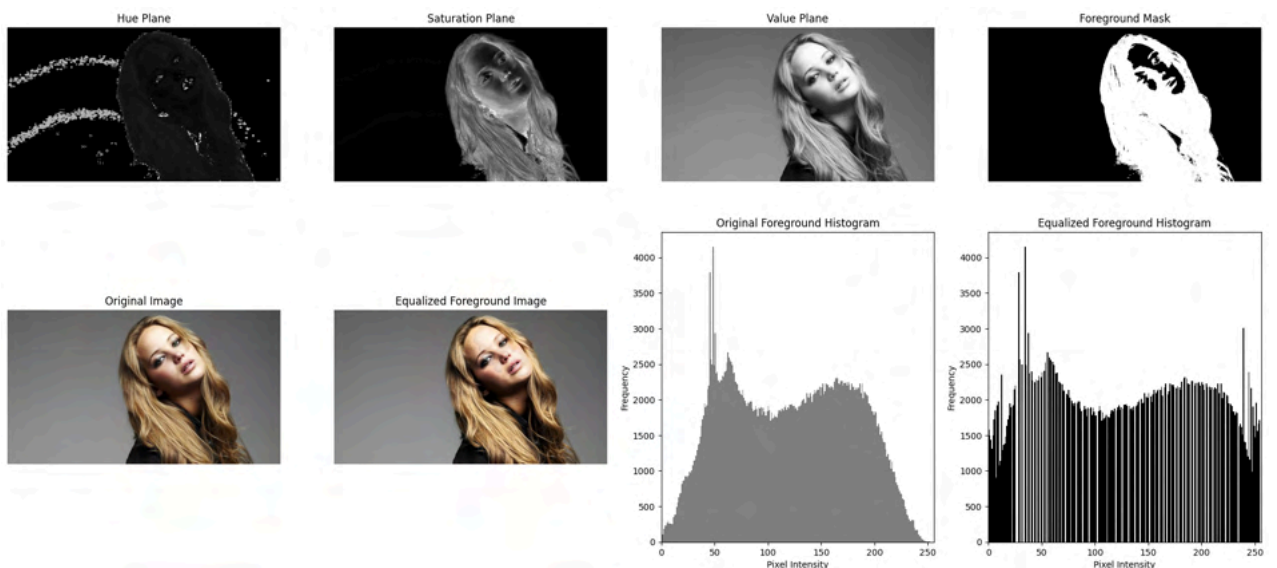
## Question 6

```
In [26]: image_path = "/content/images/q6.jpeg"
img = cv.imread(image_path)
assert img is not None
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV) #splitting
H, S, V = cv.split(hsv)
_, mask = cv.threshold(S, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU) #extract the foreground mask
V_fg = cv.bitwise_and(V, V, mask=mask) #obtaining foreground
fg_pixels = V_fg[mask == 255]
hist, _ = np.histogram(fg_pixels, 256, [0, 256])
```

```

cdf = np.cumsum(hist) #cumulative sum
# Using the formula from the slides:  $s = ((L - 1) / MN) * \sum(nj)$ 
num_fg_pixels = np.sum(mask == 255)
cdf_final = np.clip((255.0 / num_fg_pixels) * cdf, 0, 255).astype(np.uint8)
V_eq_full = cdf_final[V]
V_eq_fg = cv.bitwise_and(V_eq_full, V_eq_full, mask=mask)
eq_fg_pixels = V_eq_fg[mask == 255]
mask_inv = cv.bitwise_not(mask)
V_bg = cv.bitwise_and(V, V, mask=mask_inv) #extracting background
V_final = cv.add(V_eq_fg, V_bg) #merging with foreground
hsv_final = cv.merge([H, S, V_final])
img_final = cv.cvtColor(hsv_final, cv.COLOR_HSV2RGB)
plt.figure(figsize=(20, 10)) #displaying required figures
plt.subplot(2, 4, 1)
plt.imshow(H, cmap='gray')
plt.title("Hue Plane")
plt.axis("off")
plt.subplot(2, 4, 2)
plt.imshow(S, cmap='gray')
plt.title("Saturation Plane")
plt.axis("off")
plt.subplot(2, 4, 3)
plt.imshow(V, cmap='gray')
plt.title("Value Plane")
plt.axis("off")
plt.subplot(2, 4, 4)
plt.imshow(mask, cmap='gray')
plt.title("Foreground Mask")
plt.axis("off")
plt.subplot(2, 4, 5)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 4, 6)
plt.imshow(img_final)
plt.title("Equalized Foreground Image")
plt.axis("off")
plt.subplot(2, 4, 7)
plt.hist(fg_pixels, bins=256, range=[0, 256], color='gray')
plt.title("Original Foreground Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
plt.subplot(2, 4, 8)
plt.hist(eq_fg_pixels, bins=256, range=[0, 256], color='black')
plt.title("Equalized Foreground Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
plt.tight_layout()
plt.show()

```

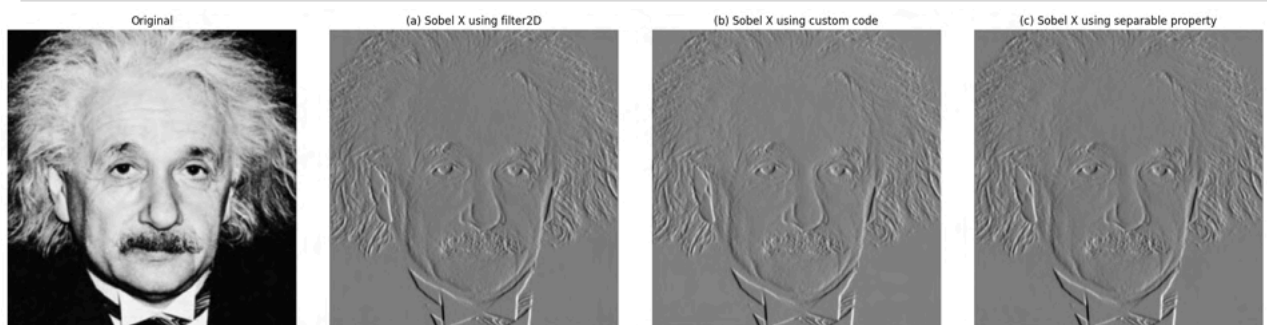


Discussion: Question 6 – Foreground-Specific Histogram Equalization

- **Mask Generation via Saturation:** The Saturation (S) plane was selected for thresholding because it provides a stark contrast between the highly saturated foreground subject and the low-saturation gray background. Applying Otsu's thresholding to this plane successfully generated a precise binary mask isolating the subject.
- **Targeted Equalization:** Histogram equalization was performed exclusively on the foreground pixels within the Value (V) channel. This was achieved by scaling the cumulative distribution function (CDF) based strictly on the count of foreground pixels rather than the entire image resolution.
- **Recombination:** The untouched background was extracted using an inverted mask and successfully recombined with the newly equalized foreground using a bitwise addition.
- **Histogram and Visual Analysis:** The original foreground histogram shows concentrated intensity peaks, while the equalized histogram displays a stretched, more evenly distributed spread across the 0–255 range. Visually, this targeted transformation significantly enhances the contrast and dynamic range of the subject's face and hair while leaving the plain gray background completely unaffected.

## Question 7

```
In [27]: image_path = "/content/images/q8.jpeg"
img = cv.imread(image_path, cv.IMREAD_GRAYSCALE)
assert img is not None
#Sobel X
kernel = np.array([[1, 0, -1],
                  [2, 0, -2],
                  [1, 0, -1]], dtype=np.float32)
grad_a = cv.filter2D(img, cv.CV_64F, kernel) #Using filter2D
pad_img = np.pad(img, ((1, 1), (1, 1)), mode='constant')
grad_b = np.zeros(img.shape, dtype=np.float64)
for i in range(img.shape[0]): #own code for SobelX
    for j in range(img.shape[1]):
        grad_b[i, j] = np.sum(pad_img[i:i+3, j:j+3] * kernel)
k_col = np.array([[1], [2], [1]], dtype=np.float32) #Using seperable property
k_row = np.array([[1, 0, -1]], dtype=np.float32)
temp = cv.filter2D(img, cv.CV_64F, k_row)
grad_c = cv.filter2D(temp, cv.CV_64F, k_col)
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
axes[0].imshow(img, cmap='gray') #figures
axes[0].set_title("Original")
axes[0].axis("off")
axes[1].imshow(grad_a, cmap='gray')
axes[1].set_title("(a) Sobel X using filter2D")
axes[1].axis("off")
axes[2].imshow(grad_b, cmap='gray')
axes[2].set_title("(b) Sobel X using custom code")
axes[2].axis("off")
axes[3].imshow(grad_c, cmap='gray')
axes[3].set_title("(c) Sobel X using separable property")
axes[3].axis("off")
plt.tight_layout()
plt.show()
```

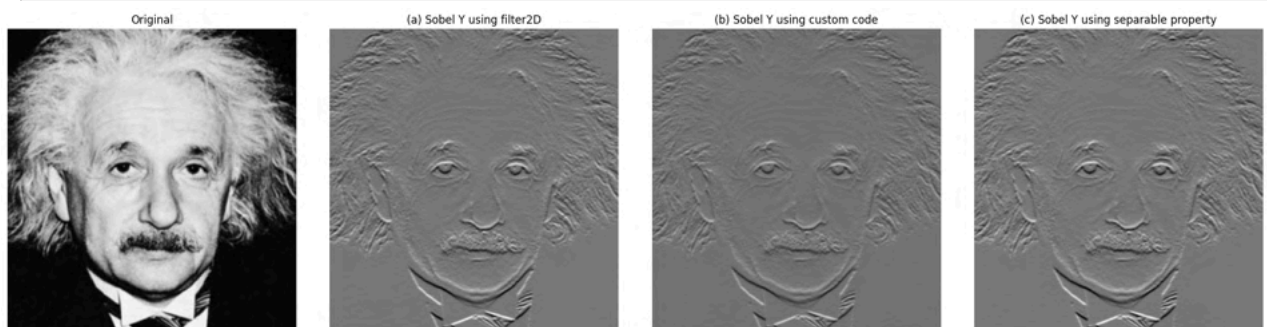


```
In [28]: #Sobel Y
kernel = np.array([[1, 2, 1],
                  [0, 0, 0],
                  [-1, -2, -1]], dtype=np.float32)
grad_a = cv.filter2D(img, cv.CV_64F, kernel) #Using filter2D
pad_img = np.pad(img, ((1, 1), (1, 1)), mode='constant')
grad_b = np.zeros(img.shape, dtype=np.float64)
for i in range(img.shape[0]): #own code for Sobel Y
```

```

for j in range(img.shape[1]):
    grad_b[i, j] = np.sum(pad_img[i:i+3, j:j+3] * kernel)
k_col = np.array([[1], [0], [-1]], dtype=np.float32) #Using seperable property
k_row = np.array([[1, 2, 1]], dtype=np.float32)
temp = cv.filter2D(img, cv.CV_64F, k_row)
grad_c = cv.filter2D(temp, cv.CV_64F, k_col)
fig, axes = plt.subplots(1, 4, figsize=(20, 5)) #figures
axes[0].imshow(img, cmap='gray')
axes[0].set_title("Original")
axes[0].axis("off")
axes[1].imshow(grad_a, cmap='gray')
axes[1].set_title("(a) Sobel Y using filter2D")
axes[1].axis("off")
axes[2].imshow(grad_b, cmap='gray')
axes[2].set_title("(b) Sobel Y using custom code")
axes[2].axis("off")
axes[3].imshow(grad_c, cmap='gray')
axes[3].set_title("(c) Sobel Y using separable property")
axes[3].axis("off")
plt.tight_layout()
plt.show()

```



## Discussion: Question 7 – Sobel Filtering and Separability

- **Directional Gradients:** The Sobel operator successfully extracts high-frequency edge information from the original image. The Sobel X kernel highlights vertical edges by computing horizontal derivatives. Conversely, the Sobel Y kernel highlights horizontal edges by computing vertical derivatives.
- **Method Equivalency:** The filtering was performed using three distinct approaches: OpenCV's `filter2D` function, a custom nested-loop spatial convolution, and utilizing the kernel's separable property. For both the X and Y directions, all three implementations produced visually identical gradient maps.
- **Advantage of Separability:** The assignment demonstrates that the  $3 \times 3$  2D Sobel kernel can be mathematically decomposed into the product of a 1D column vector and a 1D row vector. Processing the image with two sequential 1D convolutions (the separable property) yields the exact same result as the 2D convolution but requires significantly fewer mathematical operations, optimizing computational efficiency.

## Question 8

```

In [29]: def zoom_image(img, s, method): #defining the function to zoom the image
          h, w = img.shape[:2]
          new_dim = (int(w * s), int(h * s))
          if method == 'nearest': #handeling nearest neighbour
              return cv.resize(img, new_dim, interpolation=cv.INTER_NEAREST)
          elif method == 'bilinear': #handeling bilinear
              return cv.resize(img, new_dim, interpolation=cv.INTER_LINEAR)
def norm_SSD(img1, img2): #computing the normalized sum of squared difference
    if img1.shape != img2.shape:
        raise ValueError("Error ! Images must have the same dimensions")
    i1 = img1.astype(np.float64)
    i2 = img2.astype(np.float64)
    ssd = np.sum((i1 - i2)**2)
    norm = np.sum(i1**2)
    return ssd / norm
def norm_SSD_CropImage(img1, img2): #alternative for computing the normalized sum of squared difference
    if img1.shape != img2.shape:
        h = min(img1.shape[0], img2.shape[0])
        w = min(img1.shape[1], img2.shape[1])
        img1 = img1[:h, :w]

```

```

img2 = img2[:h, :w]
i1 = img1.astype(np.float64)
i2 = img2.astype(np.float64)
return np.sum((i1 - i2)**2) / np.sum(i1**2)

```

## Image 01

```

In [30]: large_img1_path = "/content/images/im01.png"
small_img1_path = "/content/images/im01small.png"
large1 = cv.imread(large_img1_path)
assert large1 is not None
small1 = cv.imread(small_img1_path)
assert small1 is not None
large1_rgb = cv.cvtColor(large1, cv.COLOR_BGR2RGB)
small1_rgb = cv.cvtColor(small1, cv.COLOR_BGR2RGB)
zoomed_nn = zoom_image(small1_rgb, 4.0, method='nearest')
zoomed_bl = zoom_image(small1_rgb, 4.0, method='bilinear')
nssd_nn = norm_SSD(large1_rgb, zoomed_nn)
nssd_bl = norm_SSD(large1_rgb, zoomed_bl)
print(f"Image 1 - Normalized SSD (Nearest-Neighbor): {nssd_nn:.6f}")
print(f"Image 1 - Normalized SSD (Bilinear): {nssd_bl:.6f}")
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
axes[0].imshow(small1_rgb)
axes[0].set_title("Small Image")
axes[0].axis("off")
axes[1].imshow(large1_rgb)
axes[1].set_title("Large Image (Original)")
axes[1].axis("off")
axes[2].imshow(zoomed_nn)
axes[2].set_title(f"Nearest-Neighbor\nNSSD: {nssd_nn:.4f}")
axes[2].axis("off")
axes[3].imshow(zoomed_bl)
axes[3].set_title(f"Bilinear\nNSSD: {nssd_bl:.4f}")
axes[3].axis("off")
plt.tight_layout()
plt.show()

```

Image 1 - Normalized SSD (Nearest-Neighbor): 0.012058

Image 1 - Normalized SSD (Bilinear): 0.010184



## Image 02

```

In [31]: large_img2_path = "/content/images/im02.png"
small_img2_path = "/content/images/im02small.png"
large2 = cv.imread(large_img2_path)
assert large2 is not None
small2 = cv.imread(small_img2_path)
assert small2 is not None
large2_rgb = cv.cvtColor(large2, cv.COLOR_BGR2RGB)
small2_rgb = cv.cvtColor(small2, cv.COLOR_BGR2RGB)
zoomed_nn_2 = zoom_image(small2_rgb, 4.0, method='nearest')
zoomed_bl_2 = zoom_image(small2_rgb, 4.0, method='bilinear')
nssd_nn_2 = norm_SSD(large2_rgb, zoomed_nn_2)
nssd_bl_2 = norm_SSD(large2_rgb, zoomed_bl_2)
print(f"Image 2 - Normalized SSD (Nearest-Neighbor): {nssd_nn_2:.6f}")
print(f"Image 2 - Normalized SSD (Bilinear): {nssd_bl_2:.6f}")
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
axes[0].imshow(small2_rgb)
axes[0].set_title("Small Image")
axes[0].axis("off")
axes[1].imshow(large2_rgb)
axes[1].set_title("Large Image (Original)")
axes[1].axis("off")
axes[2].imshow(zoomed_nn_2)
axes[2].set_title(f"Nearest-Neighbor\nNSSD: {nssd_nn_2:.4f}")
axes[2].axis("off")

```

```

axes[3].imshow(zoomed_bl_2)
axes[3].set_title(f"Bilinear\nNSSD: {nssd_bl_2:.4f}")
axes[3].axis("off")
plt.tight_layout()
plt.show()

```

Image 2 – Normalized SSD (Nearest-Neighbor): 0.004190

Image 2 – Normalized SSD (Bilinear): 0.002907



## Image 03

```

In [32]: large_img3_path = "/content/images/im03.png"
small_img3_path = "/content/images/im03small.png"
large3 = cv.imread(large_img3_path)
assert large3 is not None
small3 = cv.imread(small_img3_path)
assert small3 is not None
large3_rgb = cv.cvtColor(large3, cv.COLOR_BGR2RGB)
small3_rgb = cv.cvtColor(small3, cv.COLOR_BGR2RGB)
zoomed_nn_3 = zoom_image(small3_rgb, 4.0, method='nearest')
zoomed_bl_3 = zoom_image(small3_rgb, 4.0, method='bilinear')
print(f"Original Small Image Dimensions: {small3_rgb.shape}")
print(f"Original Large Image Dimensions: {large3_rgb.shape}")
print(f"Zoomed Image Dimensions: {zoomed_nn_3.shape}\n")
nssd_nn_3 = norm SSD_CropImage(large3_rgb, zoomed_nn_3)
nssd_bl_3 = norm SSD_CropImage(large3_rgb, zoomed_bl_3)
print(f"Image 3 – Normalized SSD (Nearest-Neighbor): {nssd_nn_3:.6f}")
print(f"Image 3 – Normalized SSD (Bilinear): {nssd_bl_3:.6f}")
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
axes[0].imshow(small3_rgb)
axes[0].set_title("Small Image")
axes[0].axis("off")
axes[1].imshow(large3_rgb)
axes[1].set_title("Large Image (Original)")
axes[1].axis("off")
axes[2].imshow(zoomed_nn_3)
axes[2].set_title(f"Nearest-Neighbor\nNSSD: {nssd_nn_3:.4f}")
axes[2].axis("off")
axes[3].imshow(zoomed_bl_3)
axes[3].set_title(f"Bilinear\nNSSD: {nssd_bl_3:.4f}")
axes[3].axis("off")
plt.tight_layout()
plt.show()

```

Original Small Image Dimensions: (365, 600, 3)

Original Large Image Dimensions: (1459, 2400, 3)

Zoomed Image Dimensions: (1460, 2400, 3)

Image 3 – Normalized SSD (Nearest-Neighbor): 0.007668

Image 3 – Normalized SSD (Bilinear): 0.005809



## Dimension Mismatch Analysis for Image 3

The output dimensions demonstrate exactly why the standard SSD calculation fails and `norm SSD_CropImage` is required:

- **Mathematical Scaling:** The small image has a height of 365 pixels. Scaling this by a factor of 4 results in a zoomed image height of exactly 1460 pixels ( $365 \times 4 = 1460$ ).
- **Reference Image Discrepancy:** The provided high-resolution reference image is only 1459 pixels tall.

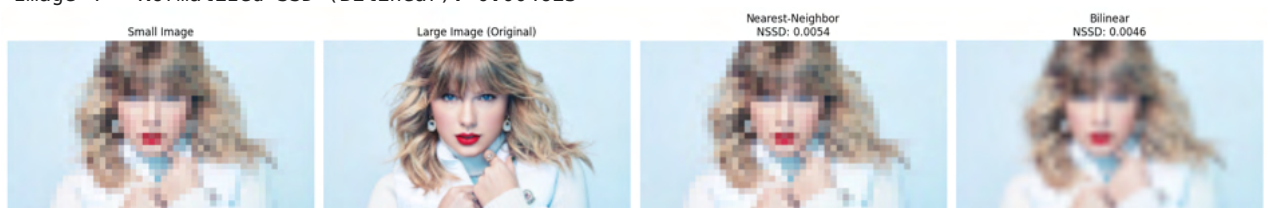
NumPy requires arrays to have identical shapes to perform the element-wise matrix subtraction (`img1 - img2`) necessary for the SSD equation. Because 1460 does not equal 1459, Python throws a `ValueError`. The custom `norm_SSD_CropImage` function resolves this by dynamically finding the minimum shared dimensions (1459, 2400) and cropping the extra row of pixels off the zoomed image before calculating the error.

## Image 04

```
In [33]: large_img4_path = "/content/images/im04.jpg"
small_img4_path = "/content/images/im04small.jpg"
large4 = cv.imread(large_img4_path)
assert large4 is not None
small4 = cv.imread(small_img4_path)
assert small4 is not None
large4_rgb = cv.cvtColor(large4, cv.COLOR_BGR2RGB)
small4_rgb = cv.cvtColor(small4, cv.COLOR_BGR2RGB)
zoomed_nn_4 = zoom_image(small4_rgb, 4.0, method='nearest')
zoomed_bl_4 = zoom_image(small4_rgb, 4.0, method='bilinear')
nssd_nn_4 = norm_SSD(large4_rgb, zoomed_nn_4)
nssd_bl_4 = norm_SSD(large4_rgb, zoomed_bl_4)
print(f"Image 4 - Normalized SSD (Nearest-Neighbor): {nssd_nn_4:.6f}")
print(f"Image 4 - Normalized SSD (Bilinear): {nssd_bl_4:.6f}")
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
axes[0].imshow(small4_rgb)
axes[0].set_title("Small Image")
axes[0].axis("off")
axes[1].imshow(large4_rgb)
axes[1].set_title("Large Image (Original)")
axes[1].axis("off")
axes[2].imshow(zoomed_nn_4)
axes[2].set_title(f"Nearest-Neighbor\nNSSD: {nssd_nn_4:.4f}")
axes[2].axis("off")
axes[3].imshow(zoomed_bl_4)
axes[3].set_title(f"Bilinear\nNSSD: {nssd_bl_4:.4f}")
axes[3].axis("off")
plt.tight_layout()
plt.show()
```

Image 4 - Normalized SSD (Nearest-Neighbor): 0.005354

Image 4 - Normalized SSD (Bilinear): 0.004623



## Discussion: Question 8 – Image Zooming and Interpolation

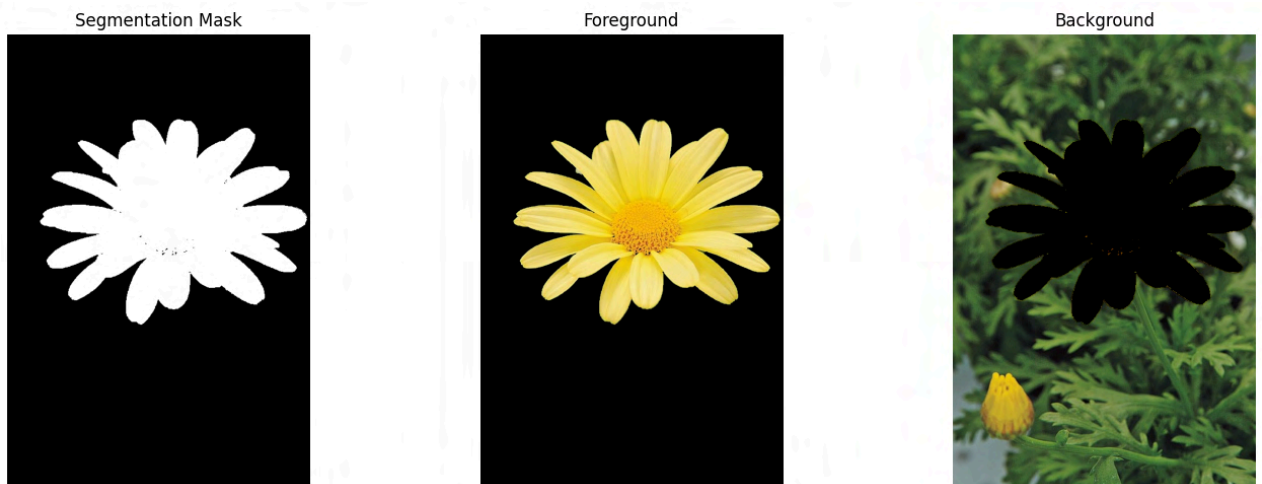
- **Implementation:** The custom `zoom_image` function successfully scales images using a given factor  $s$  by leveraging OpenCV's `cv.resize` for both nearest-neighbor (`cv.INTER_NEAREST`) and bilinear (`cv.INTER_LINEAR`) interpolation methods.
- **Quantitative Comparison (NSSD):** The Normalized Sum of Squared Differences (NSSD) was computed to evaluate the accuracy of the zoomed images against the original high-resolution ground truths. Across all four test cases, bilinear interpolation consistently produced a lower NSSD score than nearest-neighbor interpolation. For example, in Image 01, nearest-neighbor yielded an NSSD of 0.012058, while bilinear achieved 0.010184.
- **Visual Quality:** The quantitative results align with visual perception. Nearest-neighbor interpolation results in blocky, pixelated artifacts (especially noticeable along edges), whereas bilinear interpolation calculates weighted averages of surrounding pixels, producing smoother, more continuous transitions that closer resemble the original images.
- **Dimension Handling:** During the evaluation of Image 03, a shape mismatch error occurred because the mathematically scaled image (height 1460) did not perfectly match the provided reference image (height 1459).

This necessitates the use of the `norm_SSD_CropImage` function, which dynamically crops the arrays to their minimum shared dimensions before computing the SSD to prevent NumPy broadcasting errors.

## Question 9

a)

```
In [34]: image_path = "/content/images/q9.jpeg"
img = cv.imread(image_path)
assert img is not None
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
mask = np.zeros(img.shape[:2], np.uint8)
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)
h, w = img.shape[:2]
rect = (10, 10, w - 10, int(h * 0.65))
cv.grabCut(img_rgb, mask, rect, bgdModel, fgdModel, 5, cv.GC_INIT_WITH_RECT) #Using grabcut
mask2 = np.where((mask==2)|(mask==0), 0, 1).astype('uint8')
foreground = img_rgb * mask2[:, :, np.newaxis]
background = cv.subtract(img_rgb, foreground)
fig, axes = plt.subplots(1, 3, figsize=(15, 5)) #plotting
axes[0].imshow(mask2, cmap='gray')
axes[0].set_title("Segmentation Mask")
axes[0].axis("off")
axes[1].imshow(foreground)
axes[1].set_title("Foreground")
axes[1].axis("off")
axes[2].imshow(background)
axes[2].set_title("Background")
axes[2].axis("off")
plt.tight_layout()
plt.show()
```



b)

```
In [35]: blurred_background = cv.GaussianBlur(background, (45, 45), 0) #Produce an enhanced image with a sub
enhanced_img = cv.add(foreground, blurred_background)
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
axes[0].imshow(img_rgb)
axes[0].set_title("Original Image")
axes[0].axis("off")
axes[1].imshow(enhanced_img)
axes[1].set_title("Enhanced Image")
axes[1].axis("off")
plt.tight_layout()
plt.show()
```



### (c) Analysis of the Dark Halo

The dark region immediately surrounding the flower occurs due to how the background is isolated prior to blurring. When the foreground is subtracted to isolate the background, the pixels formerly occupied by the flower are set to exactly zero (pure black).

When the Gaussian blur is applied to this isolated background image, the filter averages the visible background pixels with the adjacent black pixels from the empty foreground hole. This heavily dilutes the intensity of the border pixels, causing a dark gradient or "halo" to bleed outward from the edge of the mask into the blurred background.

### Discussion: Question 9 – GrabCut Segmentation and Depth of Field Enhancement

- **GrabCut Segmentation:** The `cv.grabCut` algorithm, initialized with a bounding rectangle, successfully separated the primary yellow flower from the complex background. The resulting binary mask effectively isolated the foreground pixels, allowing for independent extraction of the subject and the background.
- **Background Processing:** To simulate a shallow depth of field, a heavy Gaussian blur was applied exclusively to the extracted background image.

### Question 10

a)

```
In [36]: image_path = "/content/images/q10.jpeg"
img = cv.imread(image_path)
assert img is not None
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
d = 9
sigma_s = 50
sigma_r = 50
cv_bilateral = cv.bilateralFilter(img_rgb, d, sigma_r, sigma_s) #Use OpenCV's bilateralFilter
gaussian_blur = cv.GaussianBlur(img_rgb, (d, d), sigma_s)
fig, axes = plt.subplots(1, 3, figsize=(18, 6)) #display figures
axes[0].imshow(img_rgb)
axes[0].set_title("Original Image")
axes[0].axis("off")
axes[1].imshow(gaussian_blur)
axes[1].set_title(f"Gaussian Blur ({d}x{d})")
```

```

axes[1].axis("off")
axes[2].imshow(cv_bilateral)
axes[2].set_title("OpenCV Bilateral Filter")
axes[2].axis("off")
plt.tight_layout()
plt.show()

```



b)

```

In [37]: def custom_bilateral_filter(img, d, sigma_r, sigma_s): # own bilateral filter
    r = d // 2
    img_pad = cv.copyMakeBorder(img, r, r, r, r, cv.BORDER_REFLECT).astype(np.float32)
    out = np.zeros_like(img, dtype=np.float32)
    y, x = np.mgrid[-r:r+1, -r:r+1]
    spatial_w = np.exp(-(x**2 + y**2) / (2 * sigma_s**2))
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            patch = img_pad[i:i+d, j:j+d]
            diff = patch - img_pad[i+r, j+r]
            color_sq_dist = np.sum(diff**2, axis=2)
            color_w = np.exp(-color_sq_dist / (2 * sigma_r**2))
            w = spatial_w * color_w
            w_sum = np.sum(w)
            for c in range(img.shape[2]):
                out[i, j, c] = np.sum(w * patch[:, :, c]) / w_sum
    return np.clip(out, 0, 255).astype(np.uint8)
custom_bilateral = custom_bilateral_filter(img_rgb, d, sigma_r, sigma_s)
def calculate_mse(img1, img2):
    return np.mean((img1.astype(np.float64) - img2.astype(np.float64)) ** 2)
mse_score = calculate_mse(cv_bilateral, custom_bilateral)
print(f"Mean Squared Error (MSE) between OpenCV and Custom Bilateral Filter: {mse_score:.4f}")
fig, axes = plt.subplots(1, 4, figsize=(24, 6))
axes[0].imshow(img_rgb)
axes[0].set_title("Original Image")
axes[0].axis("off")
axes[1].imshow(gaussian_blur)
axes[1].set_title(f"Gaussian Blur ({d}x{d})")
axes[1].axis("off")
axes[2].imshow(cv_bilateral)
axes[2].set_title("OpenCV Bilateral Filter")
axes[2].axis("off")
axes[3].imshow(custom_bilateral)
axes[3].set_title("Custom Bilateral Filter")
axes[3].axis("off")
plt.tight_layout()
plt.show()

```

Mean Squared Error (MSE) between OpenCV and Custom Bilateral Filter: 19.1746



## Discussion: Question 10 – Bilateral Filtering

- **Edge-Preserving vs. Uniform Smoothing:** The standard  $9 \times 9$  Gaussian blur applies uniform spatial smoothing across the entire image, which undesirably degrades and softens the sharp transitions along the mountain ridges. In contrast, the bilateral filter successfully smooths low-variance textural regions like the sky and water while strictly preserving the high-contrast edges of the terrain.
- **Parameter Selection:** A spatial sigma of  $\sigma_s = 50$  and a range sigma of  $\sigma_r = 50$  alongside a  $9 \times 9$  kernel ( $d = 9$ ) were utilized. These parameters apply noticeable smoothing to the background elements without bleeding pixel intensities across distinct structural boundaries.
- **Custom Implementation Validation:** Visually, the custom implementation of the bilateral filter produces results that are nearly indistinguishable from the optimized OpenCV `cv.bilateralFilter` output.
- **Quantitative Comparison:** A Mean Squared Error (MSE) of 19.1746 was calculated between the custom and OpenCV filtered images. This relatively low error score quantitatively validates the accuracy of the custom algorithm, with the minor residual variance likely stemming from underlying differences in border padding methods ( `cv.BORDER_REFLECT` ) or internal floating-point rounding during the weight matrix calculations.